

BUILD YOUR BASES FIRST

CRACKING HASH



7

LISTA DE FIGURAS

Figura 1 - Função de um Hash.....	5
-----------------------------------	---



SUMÁRIO

CRACKING HASH	4
1 O QUE SÃO HASHES	4
2 HASH NÃO É CRIPTOGRAFIA	5
3 ALGORITMOS DE HASH.....	6
4 HASHES UTILIZADOS NO WINDOWS	8
5 HASHES UTILIZADOS NO LINUX.....	9
6 QUEBRA DE HASHES.....	10
7 USO DE SALT.....	11
8 HANDS ON GERANDO HASHES NO LINUX.....	12
9 JOHN THE RIPPER E A QUEBRA DE HASHES.....	13
9.1 hands on quebrando hashes com o john the ripper	14
REFERÊNCIAS.....	16

CRACKING HASH

1 O QUE SÃO HASHES

Hashes são componentes cruciais na segurança da informação e ciência da computação em geral. Um hash é uma representação única e fixa de um conjunto de dados, geralmente de tamanho variável, como um arquivo ou uma mensagem. Esse valor, chamado de "digest" ou "hash code", é gerado por um algoritmo de hash que converte os dados em uma sequência de caracteres alfanuméricos de comprimento fixo.

A principal característica de um bom algoritmo de hash é que ele deve produzir o mesmo hash code sempre que for aplicado aos mesmos dados de entrada, tornando-o previsível e repetível. Além disso, a menor alteração nos dados de entrada deve resultar em um hash code totalmente diferente, tornando-o resistente a colisões.

Uma das aplicações mais comuns de hashes é verificar a integridade dos dados. Quando um arquivo é transferido pela Internet, um hash do arquivo original pode ser calculado e compartilhado com o destinatário. Após o recebimento do arquivo, o destinatário pode calcular o hash do arquivo recebido e compará-lo ao hash original. Se os hashes coincidirem, isso indica que o arquivo não foi alterado ou corrompido durante a transferência.

Outra aplicação importante é a autenticação de dados. As senhas armazenadas em um banco de dados não devem ser armazenadas em texto simples devido a preocupações de segurança. Em vez disso, as senhas são frequentemente transformadas em hash antes de serem armazenadas. Quando um usuário tenta fazer login, a senha inserida é transformada em hash e comparada com o hash armazenado no banco de dados.

Diversos algoritmos de hash são amplamente utilizados, como o MD5, SHA-1 (embora este seja considerado fraco hoje em dia), SHA-2 e SHA-3. A escolha do algoritmo depende dos requisitos de segurança e desempenho de uma aplicação específica.

Quer mais informações sobre os hashes MD5, SHA-1, SHA-2 e SHA-3? Então acesse: <https://www.freecodecamp.org/portuguese/news/md5-x-sha-1-x-sha-2-qual-e-o-hash-de-criptografia-mais-seguro-e-como-verifica-lo/>.

Em suma, os hashes desempenham um papel fundamental em segurança, verificação de integridade, autenticação de dados e otimização de algoritmos de busca. Eles fornecem uma maneira eficaz de representar dados de forma única e são essenciais em muitos aspectos da computação moderna.

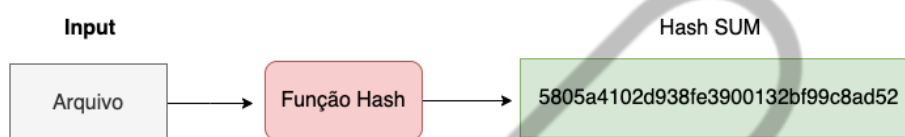


Figura 1 - Função de um Hash
Fonte: Elaborada pelo autor (2023)

2 HASH NÃO É CRIPTOGRAFIA

Hashes e criptografia são duas técnicas de segurança de dados, mas têm finalidades e características distintas.

O principal objetivo das funções hash é criar uma representação única e fixa de dados, transformando-os em uma sequência de caracteres alfanuméricos de comprimento fixo, conhecida como hash code. Isso permite a verificação de integridade e autenticidade dos dados.

As funções hash são irreversíveis, o que significa que não é possível reverter o processo de hash para recuperar os dados originais a partir do hash code. Os hashes são usados principalmente para verificar dados, não para ocultá-los. O tamanho do hash code é fixo, independentemente do tamanho dos dados de entrada. Isso significa que qualquer entrada, seja pequena ou grande, resultará em um hash de tamanho consistente.

Alguns exemplos de algoritmos de hashing incluem MD5, SHA-1, SHA-256 e HMAC. Eles são projetados para serem rápidos e eficientes na geração de hashes.

Caso queira saber mais sobre o algoritmo de hashing SHA-256, acesse: <https://academy.bit2me.com/pt/sha256-algoritmo-bitcoin/>.

Mais informações sobre o algoritmo de hashing HMAC, você encontra em: <https://acervolima.com/algoritmo-hmac-em-rede-de-computadores-1/>.

A criptografia é usada para proteger a confidencialidade dos dados, transformando-os em uma forma ilegível (cifra) que só pode ser decifrada por alguém com a chave adequada. O objetivo principal é ocultar as informações. Ao contrário do hash, a criptografia é reversível, o que significa que é possível decifrar os dados criptografados de volta para sua forma original, desde que se tenha a chave correta.

O tamanho dos dados criptografados pode variar dependendo do tamanho dos dados de entrada e do algoritmo de criptografia usado. Diferentemente dos hashes, os dados criptografados não têm um tamanho fixo. Alguns exemplos de algoritmos de criptografia incluem AES, RSA e ECC. Esses algoritmos são projetados para garantir que somente as partes autorizadas possam decifrar os dados.

Em resumo, as funções hash são usadas principalmente para verificar a integridade dos dados e criar representações únicas dos mesmos, enquanto a criptografia é usada para proteger a confidencialidade dos dados, tornando-os inacessíveis a terceiros não autorizados. Ambas são ferramentas importantes em segurança de dados, mas têm finalidades diferentes e características distintas.

3 ALGORITMOS DE HASH

Existem vários algoritmos utilizados nas funções de hash, os tipos mais comuns de funções hash utilizadas na computação são:

- **MD5 (*Message Digest Algorithm 5*):**

MD5 é um algoritmo de hashing amplamente conhecido que gera hashes de 128 bits. É rápido e produz hashes de tamanho fixo. No entanto, é considerado fraco para fins de segurança, pois não é resistente a colisões (duas entradas diferentes podem resultar no mesmo hash).

O MD5 é comumente usado em checksums e verificação de integridade de arquivos, mas não é recomendado para aplicações de segurança.

- **SHA-1 (Secure Hash Algorithm 1):**

SHA-1 é outro algoritmo de hashing amplamente utilizado, que gera hashes de 160 bits. Embora seja mais seguro que o MD5, o SHA-1 também é vulnerável a colisões e não é mais considerado seguro para aplicações críticas de segurança. O SHA-1 foi usado em certificados digitais e assinaturas, mas seu uso foi desencorajado devido a preocupações com a segurança.

- **SHA-256 (Secure Hash Algorithm 256):**

O SHA-256 faz parte da família SHA-2 e gera hashes de 256 bits. É considerado seguro e é amplamente usado em aplicativos de segurança, como certificados digitais, assinaturas eletrônicas e autenticação de dados. O SHA-256 é amplamente utilizado em criptografia moderna e é uma escolha comum para garantir a integridade e a segurança dos dados.

- **HMAC (Hash-based Message Authentication Code):**

O HMAC não é um algoritmo de hash por si só, mas um método para calcular hashes autenticados usando algoritmos de hash convencionais (como SHA-256 ou MD5) juntamente com uma chave secreta. O HMAC fornece autenticação e integridade dos dados, garantindo que os dados não tenham sido alterados e que a origem seja autenticada. É usado em autenticação de mensagens e comunicação segura, como na autenticação de tokens em sistemas de autenticação de dois fatores.

- **CRC32 (Cyclic Redundancy Check):**

O CRC32 é uma técnica de verificação de integridade que gera hashes de 32 bits. É rápido e eficiente em detecção de erros, mas não é projetado para fins de segurança. É mais comumente usado em comunicações de rede e armazenamento de dados para detecção de erros. O CRC32 é usado em protocolos de comunicação, como Ethernet e ZIP, para verificar a integridade dos dados transmitidos.

Cada um desses tipos de hash tem suas próprias características e finalidades específicas. É importante escolher a função hash apropriada com base nos requisitos de segurança e desempenho da sua aplicação. Para segurança robusta, geralmente é recomendável usar algoritmos de hashing mais seguros, como SHA-256, em vez de MD5 ou SHA-1.

4 HASHES UTILIZADOS NO WINDOWS

O Windows utiliza vários tipos de hashes para diferentes finalidades de segurança e autenticação. A seguir estão alguns dos hashes mais comuns usados no Windows:

- **LM Hash (LAN Manager Hash):**

Foi historicamente usado para armazenar senhas de contas de usuário no Windows. No entanto, é considerado extremamente fraco e inseguro, pois divide a senha em duas metades de 7 caracteres e aplica hash separadamente.

O uso do LM Hash foi descontinuado em versões mais recentes do Windows, mas ainda pode estar presente em sistemas mais antigos.

- **NTLM Hash (NT LAN Manager Hash):**

É uma versão mais segura do hash usada para armazenar senhas de contas de usuário no Windows. O NTLM Hash é mais resistente a ataques de força bruta em comparação com o LM Hash.

O NTLM Hash é usado para autenticação de usuário em redes Windows e é comumente usado em ambientes corporativos.

- **NTLMv2 Hash:**

É uma versão ainda mais segura do NTLM Hash e é usada em versões mais recentes do Windows. O NTLMv2 Hash é usado para autenticação de usuário em ambientes Windows e é mais seguro em comparação com o NTLM Hash original.

- **SAM (Security Accounts Manager) Hashes**:**

O SAM é um banco de dados de contas de usuário e suas informações de autenticação no Windows. As senhas no SAM são armazenadas como hashes, mas o tipo de hash usado pode variar com base na versão do Windows e nas configurações de segurança. O SAM Hash é usado para autenticação local de usuário no Windows.

É importante notar que as versões mais recentes do Windows tendem a usar métodos de autenticação mais seguros e hashes mais fortes. O uso de hashes fracos, como o LM Hash, é desencorajado em ambientes modernos devido a preocupações

com a segurança. Para garantir a segurança das senhas e da autenticação, é essencial manter o sistema atualizado e adotar práticas recomendadas de segurança.

5 HASHES UTILIZADOS NO LINUX

No Linux, os hashes de senha também variam dependendo da distribuição e da configuração do sistema. No entanto, o método mais comum e amplamente utilizado para armazenar hashes de senhas no Linux é o uso do algoritmo de hash "crypt" baseado no DES (Data Encryption Standard) e suas variantes, bem como o uso do algoritmo de hash SHA-512. Abaixo, explicarei esses dois métodos principais:

- **Algoritmo de Hash "crypt" (DES e Variantes):**

Este é o método mais antigo e amplamente suportado de armazenamento de senhas no Linux. O algoritmo "crypt" usa uma técnica de hashing de senha baseada no DES e produz hashes de 13 a 24 caracteres. É considerado relativamente seguro, mas não é tão resistente a ataques de força bruta quanto algoritmos mais modernos. Senhas no formato "crypt" são armazenadas em arquivos como o "/etc/shadow" no Linux.

- **Algoritmo de Hash SHA-512:**

O SHA-512 é uma implementação do algoritmo SHA-2 e é uma opção mais segura e moderna para armazenar senhas no Linux. Ele gera hashes de 86 caracteres, proporcionando maior resistência a ataques de força bruta em comparação com o "crypt".

Muitas distribuições Linux modernas, como Ubuntu, Fedora e CentOS, adotaram o SHA-512 como método de hash de senha padrão.

Além disso, é importante mencionar que o Linux permite que você escolha diferentes algoritmos de hash para armazenar senhas, dependendo das configurações do sistema e das políticas de segurança. Outros algoritmos, como bcrypt e Argon2, também são utilizados em algumas configurações para melhorar a segurança das senhas.

Geralmente, as senhas e seus hashes correspondentes são armazenados em arquivos de senha protegidos, como `/etc/shadow`. Esses hashes são usados para autenticar os usuários durante o processo de login e garantir a segurança das senhas armazenadas. É importante manter senhas seguras e usar algoritmos de hash adequados para proteger as informações dos usuários em sistemas Linux.

6 QUEBRA DE HASHES

Hashes podem ser quebrados, mas a dificuldade de fazê-lo depende de vários fatores, incluindo o algoritmo de hash usado, o tamanho e complexidade da senha original e os recursos computacionais disponíveis. Além disso, a quebra de um hash pode ser feita de várias maneiras, incluindo ataques de força bruta, ataques de dicionário e o uso de "rainbow tables".

Uma *Rainbow Table* (tabela arco-íris) é uma técnica usada para acelerar ataques de quebra de hash, especialmente quando senhas fracas ou previsíveis são usadas. Ela é uma tabela precomputada que contém uma grande quantidade de hashes e suas correspondentes entradas originais (senhas). A ideia é que, em vez de calcular hashes repetidamente para tentar encontrar uma correspondência, um atacante possa pesquisar a tabela arco-íris para encontrar uma correspondência pré-calculada.

Aqui está um exemplo simplificado de como uma Rainbow Table funciona:

1. Um atacante pré-calcula hashes para uma variedade de senhas possíveis e armazena essas correspondências em uma tabela arco-íris.
2. Quando o atacante obtém acesso a um hash de senha alvo (por exemplo, a partir de um arquivo de senhas vazadas), ele consulta a tabela arco-íris em busca de uma correspondência.
3. Se uma correspondência for encontrada, o atacante conhecerá a senha original associada ao hash.

É importante observar que o uso de hashes com salts (um valor aleatório único adicionado às senhas antes de calcular o hash) pode tornar as tabelas arco-íris ineficazes, pois cada senha será única mesmo que seja a mesma senha base. Além

disso, o uso de algoritmos de hash seguros, como SHA-256 ou bcrypt, pode tornar muito mais difícil a quebra de hashes, mesmo com tabelas arco-íris.

Em resumo, as hashes podem ser quebradas, especialmente quando senhas fracas ou previsíveis são usadas, e as tabelas arco-íris são uma técnica que os atacantes podem usar para acelerar esse processo. Portanto, é fundamental usar senhas fortes, algoritmos de hash seguros e, sempre que possível, adicionar salts para proteger as senhas contra ataques de quebra de hash.

7 USO DE SALT

Um salt é um valor aleatório único que é gerado e adicionado às senhas antes de calcular o hash. A principal finalidade do salt é aumentar a segurança das senhas armazenadas e tornar mais difícil a quebra de hashes por meio de ataques de dicionário ou tabelas arco-íris.

A seguir algumas vantagens na utilização de salt:

- **Impede colisões:** Um salt garante que senhas idênticas tenham hashes diferentes. Isso ocorre porque o salt é aleatório e único para cada usuário ou senha, mesmo que as senhas reais sejam iguais. Como resultado, as colisões (quando duas entradas diferentes geram o mesmo hash) são muito menos prováveis.
- **Dificulta Ataques de Dicionário e Rainbow Tables:** Sem salts, um atacante poderia pré-calcular hashes para senhas comuns e, em seguida, compará-los com os hashes armazenados. Com salts exclusivos para cada senha, esse tipo de ataque é ineficaz, pois o atacante precisaria pré-calcular hashes para cada possível salt.
- **Aumenta a complexidade das senhas:** O uso de salts torna as senhas mais complexas, pois um atacante precisa não apenas adivinhar a senha, mas também o salt correspondente.

Um exemplo simplificado de como o salt funciona:

1. Um usuário cria uma senha, como "minhaSenha".
2. O sistema gera um salt aleatório exclusivo, como "abc123".

3. O salt é adicionado à senha do usuário, criando algo como "minhaSenhaabc123".

4. O sistema calcula o hash da senha com o salt incluído, armazenando o hash resultante, bem como o salt, no banco de dados.

Portanto, quando o usuário tenta fazer login, o sistema utiliza o mesmo salt para a senha fornecida, calcula o hash e verifica se ele corresponde ao hash armazenado. Como o salt é exclusivo para cada usuário ou senha, mesmo senhas idênticas gerarão hashes diferentes. O uso de salts é uma prática recomendada para aumentar a segurança das senhas armazenadas e tornar os sistemas menos vulneráveis a ataques de quebra de hash.

8 HANDS ON GERANDO HASHES NO LINUX

No Linux, você pode gerar hashes de senhas diretamente no terminal usando comandos como ``openssl``, ``sha256sum``, ``md5sum``, entre outros. Aqui estão alguns exemplos de como gerar hashes de senhas no terminal do **Kali Linux**:

Gerando um Hash MD5:

```
#echo -n "suaSenha" | md5sum
```

Gerando um Hash SHA-256:

```
#echo -n "suaSenha" | sha256sum
```

Gerando um Hash SHA-512:

```
#echo -n "suaSenha" | sha512sum
```

Lembre-se de substituir "suaSenha" pela senha que você deseja hashear. O uso da opção **-n** com **echo** garante que não haja uma quebra de linha (`\n`) no final da senha, o que poderia afetar o hash.

Depois de executar o comando, o hash será exibido no terminal como uma sequência hexadecimal. Você pode copiar e colar esse hash, se necessário.

Por exemplo, ao executar o comando **echo -n "minhaSenha" | md5sum**, você obterá um hash MD5 semelhante a `'e10adc3949ba59abbe56e057f20f883e'`.

9 JOHN THE RIPPER E A QUEBRA DE HASHES

O John the Ripper é uma ferramenta de software amplamente conhecida e usada para testar a segurança de senhas por meio da quebra de hashes de senhas. Essa ferramenta é comumente usada por profissionais de segurança cibernética, testadores de penetração e administradores de sistemas para avaliar a força das senhas e identificar vulnerabilidades em sistemas e redes.

A principal função do John the Ripper é realizar ataques de força bruta, ataques de dicionário e outros tipos de ataques contra hashes de senhas. Ele tenta decifrar senhas recuperando a senha original correspondente a um hash de senha conhecido. O John the Ripper é conhecido por ser altamente configurável e suporta uma variedade de algoritmos de hash, incluindo os usados no Unix/Linux, Windows e outros sistemas operacionais.

Algumas características notáveis do John the Ripper incluem:

- 1. Suporte a vários algoritmos:** Ele suporta uma ampla variedade de algoritmos de hash, incluindo DES, MD5, SHA-1, SHA-256, bcrypt, entre outros.
- 2. Ataques personalizáveis:** Os usuários podem personalizar os ataques, ajustando os parâmetros para atender às necessidades específicas de teste de segurança.
- 3. Dicionários e regras:** Além de ataques de força bruta, o John the Ripper permite o uso de dicionários de palavras e regras para gerar combinações de senhas com base em padrões comuns.
- 4. Análise de senhas fracas:** Ele pode identificar senhas fracas e comuns rapidamente, ajudando os administradores de sistemas a melhorar a segurança de suas implementações.

É importante mencionar que o John the Ripper é uma ferramenta legítima, frequentemente usada por profissionais de segurança para fins de teste e auditoria. No entanto, pode ser usado de maneira maliciosa por indivíduos mal-intencionados para quebrar senhas ilegalmente. Portanto, é fundamental usá-lo com ética e apenas em sistemas e redes para os quais você tenha autorização ou consentimento para realizar testes de segurança.

9.1 hands on quebrando hashes com o john the ripper

Se você deseja usar o John the Ripper para quebrar um hash MD5, o processo é semelhante ao que descrevi anteriormente, mas você deve especificar o formato do hash como "md5" e fornecer um arquivo com hashes no formato apropriado. Aqui está um exemplo de como fazer isso utilizando o Kali Linux:

1. Crie um arquivo com o Hash MD5:

No Kali Linux, crie um arquivo (por exemplo, "hashes.txt") com os hashes MD5 que deseja quebrar. Cada linha do arquivo deve conter um único hash MD5. Por exemplo:

```
098f6bcd4621d373cade4e832627b4f6
e10adc3949ba59abbe56e057f20f883e
5f4dcc3b5aa765d61d8327deb882cf99
```

2. Execute o John the Ripper:

Use o John the Ripper para realizar um ataque de dicionário, fornecendo o arquivo com os hashes e o dicionário de palavras. O comando pode ser assim:

```
john --format=raw-md5 --wordlist=dicionario.txt hashes.txt
```

raw-md5: Especifica o formato do hash como MD5.

dicionario.txt: Substitua pelo caminho para o seu arquivo de dicionário de palavras.

hashes.txt: Substitua pelo caminho para o arquivo com os hashes MD5.

3. Visualizar os Resultados:

Após o John the Ripper concluir a quebra da senha (se bem-sucedido), você pode ver as senhas quebradas executando o seguinte comando:

```
john --show hashes.txt
```

Isso exibirá as senhas que foram quebradas com sucesso.

Lembre-se de que a quebra de senhas deve ser realizada de maneira ética e em conformidade com todas as leis e regulamentos aplicáveis. O uso indevido dessa

ferramenta pode ser ilegal e antiético. Certifique-se de obter a devida autorização antes de realizar testes de segurança em sistemas e redes.

EMSE

REFERÊNCIAS

ZOCHIO, F. M. **Introdução a Criptografia**. São Paulo: Editora Novatec, 2019.

STALLINGS, W. **Criptografia e segurança de redes princípios e práticas**. São Paulo: Editora Pearson, 2014.

EMANIP